Deep Learning for Medical Image Segmentation: A Focus on U-Net for Brain MRI Tumor Detection and Classification

Om Patel

Rocky Point High School

Introduction:

Medical image segmentation is a crucial task in medical image analysis, which involves identifying and segmenting regions of interest in medical images. Deep learning approaches have shown promising results in this field, and U-Net is one such approach that has been widely used for medical image segmentation. U-Net is a convolutional neural network that was originally introduced for biomedical image processing, where segmentation accuracy is very important ((Malhotra et al., 2022). The U-NET Brain-MRI-Segmentation is a project that uses U-Net with TensorFlow and Keras for brain MRI segmentation. The dataset used in this project is a popular one in Kaggle called Brain MRI Segmentation, which contains brain MRI images together with manual flare abnormality segmentation masks. Deep learning methods have yielded outstanding results in various application domains, including medical image analysis. However, medical image segmentation is a cross-disciplinary field that requires collaboration between clinicians and artificial intelligence scientists. The design of the network structure, 3D data segmentation model design, and loss function design are also important aspects to consider in deep learning approaches for medical image segmentation. This research aims to use Computer Vision and Deep Learning to examine splices of magnetic resonance images and locate tumors autonomously with greater efficiency.

The article discusses the importance of medical image segmentation in medical image analysis, which involves identifying and segmenting regions of interest in medical images. The article highlights the use of deep learning approaches, particularly U-Net, for medical image segmentation. The U-NET Brain-MRI-Segmentation project is introduced, which uses U-Net with TensorFlow and Keras for brain MRI segmentation. The article emphasizes the importance of collaboration between clinicians and artificial intelligence scientists in the field of medical

image segmentation (Suganyadevi, 2022). The design of the network structure, 3D data segmentation model design, and loss function design are also important aspects to consider in deep learning approaches for medical image segmentation. The article concludes by stating that deep learning methods have yielded outstanding results in various application domains, including medical image analysis, and that this research aims to use Computer Vision and Deep Learning to examine splices of Magnetic resonance images and locate tumors autonomously (Suganyadevi, 2022).

Mostafa's article explores the use of deep learning, specifically convolutional neural networks (CNNs), for the diagnosis and segmentation of brain tumors (Mostafa, 2023). It emphasizes the challenges in brain tumor diagnosis, the need for early detection, and the potential of deep learning techniques to improve the accuracy and efficiency of the process. The article discusses the Brain Tumor Segmentation (BraTS) dataset, which contains 335 annotated MRI images for model development. A deep CNN model is trained on this dataset, achieving an impressive validation accuracy of 98%. Background information on brain tumors, their classification, symptoms, and treatment options is provided. The article also highlights the importance of feature extraction and the quantity of training data in determining model performance, along with the use of ensemble configurations to enhance classification accuracy (Mostafa, 2023). The proposed framework for brain tumor identification using deep learning methods is explained, and the article concludes by underscoring the promise of deep learning in medical image analysis and suggesting avenues for future research, including optimizing model hyperparameters and exploring real-world applications and additional datasets. The article's discussion of deep learning and convolutional neural networks (CNNs) for brain tumor segmentation is highly relevant to the topic of medical image segmentation. It underscores the

significance of accurate segmentation in medical image analysis, which is crucial for identifying and isolating regions of interest in medical images like brain MRIs. Specifically, it mentions the use of the U-Net architecture, a widely adopted deep learning approach for medical image segmentation, and its application to brain MRI segmentation using the Brain MRI Segmentation dataset. This aligns with the idea that deep learning methods, including U-Net, have demonstrated exceptional results in medical image analysis. The article's emphasis on the collaboration between clinicians and artificial intelligence scientists, network structure design, 3D data segmentation, and loss function design further emphasizes the interdisciplinary nature and complexity of medical image segmentation. Ultimately, this research, which employs computer vision and deep learning to autonomously detect tumors in MRI images, aligns with the broader context of advancements in deep learning techniques for medical image segmentation and collaborative efforts in this field. (Mostafa, 2023).

This article discusses a lightweight implementation of U-Net, a deep learning architecture, for brain tumor segmentation in Magnetic Resonance Imaging (MRI) scans. The primary focus is on real-time segmentation, and the proposed model requires a minimal amount of training data and doesn't need additional data augmentation. The paper reports promising results with a mean intersection-over-union (IoU) of 89%, outperforming standard benchmark algorithms (Walsh et al.,2022). The work also emphasizes the use of three perspective planes for simplified segmentation and explores various techniques for brain tumor segmentation in medical images. The article outlines the data preprocessing, implementation details, and the specific dataset used. It also describes U-Net and its architecture, highlighting its role in efficiently segmenting images, particularly when the input image and the binary mask have matching dimensions. The provided article closely aligns with the topic of medical image

segmentation, particularly in the context of brain MRI analysis. It highlights the importance of accurate segmentation in medical imaging and discusses the application of deep learning, specifically the U-Net architecture, for segmenting brain MRI scans, with a focus on tumor localization. The article utilizes a dataset containing brain MR images and manual segmentation masks, emphasizing the significance of data in this field. Additionally, it touches on the cross-disciplinary nature of medical image segmentation, where collaboration between medical experts and AI scientists is essential. The article's focus on U-Net model design and structure complements the broader theme of using deep learning for autonomous tumor detection in MRI images. (Walsh et al., 2022).

This article explores the application of deep learning techniques, particularly Convolutional Neural Networks (CNNs), for the segmentation of brain tumors in medical images, with a primary focus on Magnetic Resonance Imaging (MRI). Traditionally performed by radiologists, manual segmentation has limitations in terms of time consumption and inter-rater variability (Magadza and Serestina, 2021). Deep learning algorithms have gained attention for their potential to automate this process by learning from large datasets. The article reviews various deep learning methods and architectures, including UNet-based models and ensemble methods, and discusses their potential for improving brain tumor segmentation accuracy. Despite significant advancements, the article points out that the robustness of these methods still falls short of expert radiologist performance, highlighting the need for improved preprocessing and addressing class imbalance. Additionally, the lack of a large-scale medical training dataset remains a significant challenge in enhancing segmentation algorithm performance. Overall, the article aims to leverage deep learning to better understand brain tumor characteristics and improve treatment planning in the field of medical image analysis. The mentioned article on

brain tumor segmentation using deep learning techniques is highly relevant to your topic of medical image segmentation, particularly in the context of brain MRI. It aligns with the idea that deep learning approaches, including the U-Net architecture, have demonstrated promise in medical image segmentation tasks. The article emphasizes the importance of improving the accuracy of segmentation, which is crucial in medical image analysis. It also discusses the challenges related to the design of network structures, 3D data segmentation models, and loss functions, which are critical aspects in the field of medical image segmentation. Furthermore, the article highlights the need for collaboration between clinicians and artificial intelligence scientists, emphasizing the interdisciplinary nature of this field, which is in line with your research's goal to use computer vision and deep learning for autonomous tumor detection in MRI images. Additionally, the use of brain MRI and the mention of datasets such as the Kaggle Brain MRI Segmentation dataset reinforces the relevance of the discussed approaches to your research in medical image segmentation. (Magadza and Serestina, 2021).In conclusion, the articles collectively underscore the critical role of medical image segmentation in the field of medical image analysis, emphasizing the identification and segmentation of regions of interest in various medical images, particularly in the context of brain MRI scans. Deep learning approaches, with a notable focus on the U-Net architecture, have shown promising results in this domain. The articles emphasize the importance of accurate segmentation, address the need for sufficient training data, and delve into network structure design, 3D data segmentation models, and loss function design. Each work contributes valuable insights into advancing deep learning techniques for medical image segmentation, showcasing their potential applications, and highlighting areas for improvement. Despite the advancements, challenges such as class imbalance, limited large-scale medical training datasets, and the quest for improved

preprocessing persist. The articles collectively stress the ongoing efforts to optimize segmentation algorithms and enhance their robustness, with the ultimate goal of improving treatment planning and understanding medical conditions. In the broader context of medical image segmentation, the research discussed aligns with the overarching aim of leveraging computer vision and deep learning for autonomous and quicker tumor detection in MRI images. As the field continues to evolve, addressing these challenges and refining deep learning methodologies for medical image segmentation will likely lead to further breakthroughs, ultimately benefiting the diagnosis and treatment of various medical conditions through improved and automated image analysis techniques.

Methods:

1. *Software and Hardware setup*

To start developing such a program, you will need a few items and software programs. First, you will need a computing device like a laptop or desktop to code on. The second thing you will need is Visual Studio Code (VS Code), an IDE allowing users to code which can be installed using this website: https://code.visualstudio.com. If you are more familiar with Jupyter Notebook website, you can use that, but this program focuses on the use of VS Code. After, you will need to install the latest version of Python using this link: https://python.org/. Next, you will need a command-line interface, Terminal for Mac, Command Prompt for Windows, or any 3rd party command-line interface like Cmder or Conda will also work. Another item you will need is a web browser that you have unrestricted access to.

2. *Installation and Setup*

Creating a Visual Studio Code Jupyter project involves several steps to ensure seamless integration of Jupyter notebooks within the VS Code environment. Begin by installing Visual Studio Code on your machine using the link above, if you haven't already. Once installed, open VS Code and navigate to the Extensions view by clicking on the Extensions icon in the Activity Bar on the side of the window. Search for the "Jupyter" and “Python” extensions and install them. The Jupyter extension provides Jupyter Notebook support in VS Code and the Python extension installs all dependencies needed to run Python. After installation, reload the window to activate the extensions. Next, create a new folder on your computer for your Jupyter project, name it something easy to remember and meaningful, and open it in VS Code. Then, click on “New File” and if it’s an option click on “Jupyter Notebook .ipynb Support”. If it’s not an option, click on “Python File” and press “Command (on Mac)/ Control(on Windows) + Shift + P” to open up the command palette. In the command palette, type Create: New Jupyter Notebook and click it. Now Jupyter Notebook is installed on VS Code. Jupyter Notebook in VS Code provides a rich set of features, including code execution, interactive widgets, and inline visualization. The next phase in the setup process is the Brain MRI image slices. Go to this link: https://www.kaggle.com/datasets/mateuszbuda/lgg-mri-segmentation, sign up for a Kaggle account and download the dataset. When looking at one of the slices, the file name is TCGA_CS_5393_19990606_13_mask.tif. The “TCGA” represents what institution the MRIs were taken at. The “CS_5393_19990606” represents the patient’s ID, and the “13_mask.tif” is the slice number of the patient’s brain followed by the file type. The final step in the setup

process is to install the packages that you will need for Machine Learning, Computer Vision, and Graphical Analysis. *Figure 1* shows the following packages that need to be installed using VS Code in the cells. This should be done by using the "pip install" command (pip install (package_name)). Use the command "!pip freeze" to check the packages that are currently installed. After installing a package, restart the kernel by clicking "restart" next to "Run All". If you do not see the "restart button" try running the program first. *Figure 2* shows how to install the packages and important notes. Only some of the packages are shown there, so make sure you use the "!pip freeze" command and use Google to install the packages you still need by typing "How to install "package_name" using pip" and clicking on official links like python.org, Stack Overflow, GeeksforGeeks, or other reliable sources. Be sure you see "successfully installed" on the output screen when installing the package, and make sure to restart the kernel after installing each package (red and yellow ovals in Figure 2).

```
import os
import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use("ggplot")
%matplotlib inline

import cv2
from tqdm import tqdm_notebook, trange
from glob2 import glob
from itertools import chain
from skimage.io import imread, imshow, concatenate_images
from skimage.transform import resize
from skimage.morphology import label
from sklearn.model_selection import train_test_split

import tensorflow as tf
from skimage.color import rgb2gray
from tensorflow.keras import Input
from tensorflow.keras.models import Model, load_model, save_model
from tensorflow.keras.layers import Input, Activation, BatchNormalization, Dropout, Lambda, Conv2D, Conv2DTranspose, MaxPooling2D, concatenate
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

from tensorflow.keras import backend as K
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

Figure 1: shows the following dependencies that will be used for most of the project (two more packages will be installed later because it is not needed at the moment).

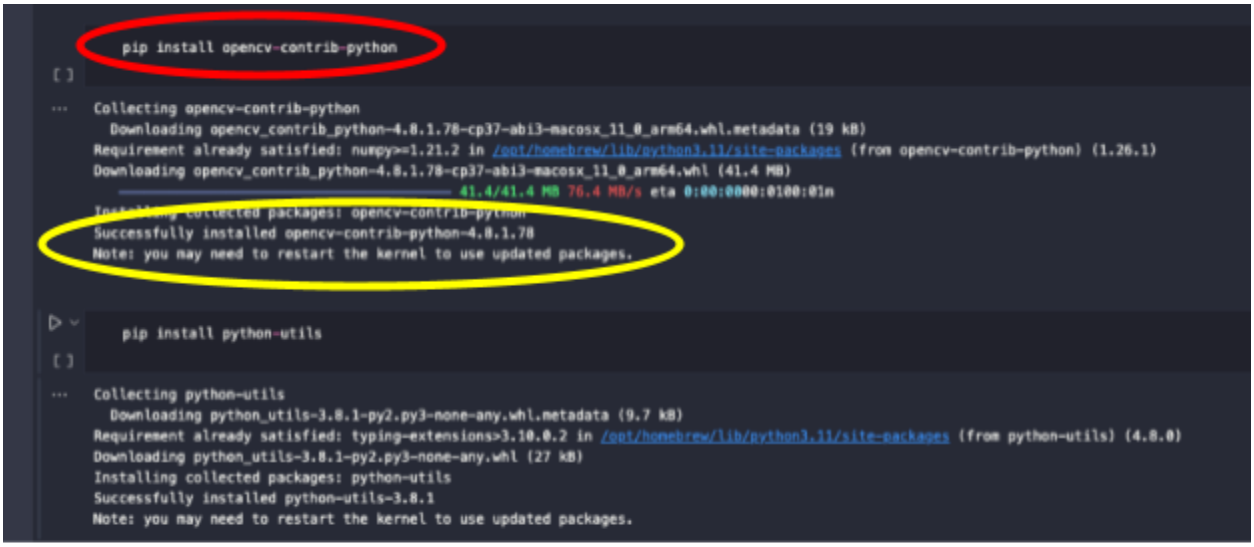


Figure 2: shows the package *OpenCV* and *Utils* being installed in the cells of VSCode.

3. *Finding the dataset:*

After installing all the packages, we need to write code to find the Brain MRI slices in your computer (*Figure 3*). The path that is highlighted in yellow in *Figure 3* should be where the MRI slice dataset we downloaded from Kaggle. Find where you installed the dataset, right-click on it, and press "Copy". Then type into the VSCode cell "mask_files = glob(" and then press "Command/ Control + V"")". This will paste the path of where the folder is stored on your computer (this path is shown in macOS format and it may look different on Windows, but the steps still work). The output should be all the files with the keyword "mask" at the end with the number 3929 which is the correct number of image slices (*Figure 3).*

```
# Setting size parameters of images
im_weidth = 256
im_height = 256
[29]  ✓ 0.0s                                                    Python
```

Loading the images and mask paths

```
image_filenames_trains = []

#creating a list of all files containing the word 'mask'
mask_files = glob('../Users/om_patel/Downloads/Brain_MRI/lgg-mri-segmentation/kaggle_3m/*/*_mask*')


for i in mask_files:
    image_filenames_trains.append(i.replace('_mask', ''))

print(image_filenames_trains[:10])
len(image_filenames_trains)
[30]  ✓ 0.0s                                                    Python
...  ['../Users/om_patel/Downloads/Brain_MRI/lgg-mri-segmentation/kaggle_3m/TCGA_CS_6667_20011105/TCGA_CS_6667_20011105_8.tif', '../Users/

3929
```

Figure 3: shows the code needed to find the slices in your computer

4. *Dice Loss/Coefficient Explanation:*

The Dice coefficient is a measure of the similarity between two samples. In the context of image segmentation, it is used to evaluate the performance of an algorithm by comparing the overlap between the predicted and ground truth segmentation masks. The Dice coefficient ranges from 0 to 1, where 1 indicates perfect overlap and 0 indicates no overlap. In the context of image segmentation, the predicted mask and ground truth mask (Table 1) refer to binary images that represent the regions of interest in an image (*Figure 4A)*. Below are detailed differences between the two. During the training process, the model is presented with input images, and the loss between the predicted masks and the ground truth masks is computed using metrics like the Dice coefficient or other segmentation loss functions. The model then adjusts its parameters through

backpropagation to minimize this loss, improving its ability to accurately segment objects in new, unseen images. The Dice loss is a loss function derived from the Dice coefficient. It is used as an objective function during the training of segmentation models. Minimizing the Dice loss during training encourages the model to produce segmentation masks that closely match the ground truth (*Figure 4B).* The closer the Dice Loss is to 0, the better the alignment between the predicted and ground truth masks. The subtraction of 1 in the Dice Loss function is a design choice and is commonly used in the context of loss functions. The Dice Coefficient measures the similarity between two sets, with a value of 1 indicating perfect overlap and a value of 0 indicating no overlap. Therefore, when using the Dice Coefficient as a basis for a loss function, subtracting it from 1 creates a loss function that is minimized when the Dice Coefficient is maximized, and vice versa.

Ground Truth Mask:	*Predicted Mask:*
The ground truth mask is the manually annotated or labeled segmentation mask that indicates the correct segmentation of objects or regions in an image.	-The predicted mask is the output of a segmentation model, representing the model's estimation of the regions or objects in the input image.

Ground Truth Mask:	*Predicted Mask:*
Each pixel in the ground truth mask is assigned a label, typically 1 for the object of interest and 0 for the background.	Similar to the ground truth mask, the predicted mask is a binary image where each pixel is assigned a label, with 1 indicating the predicted object and 0 indicating the predicted background.
Ground truth masks are used during the training of a segmentation model to provide a target for the model to learn from.	The goal during training is to optimize the model so that its predicted masks closely match the ground truth masks.

(Table 1): shows the main differences between the term *ground truth mask* and *predicted mask*.

$$Dice(X,Y) = \frac{2 * |X \cap Y|}{|X| + |Y|}$$

(Figure 4A): shows the formula for the Dice coefficient where X is the predicted binary mask. Y is the ground truth binary mask. $|X \cap Y|$ is the intersection of the predicted and ground truth masks. $|X|$ is the total number of pixels in the predicted mask. $|Y|$ is the total number of pixels in the ground truth mask.

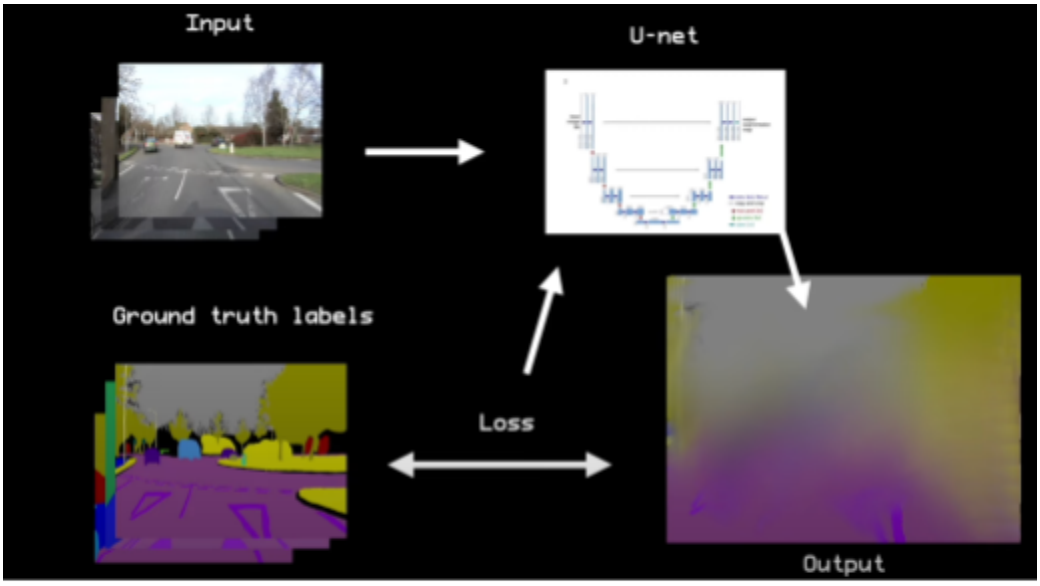
$$DiceLoss(X,Y) = 1 - \frac{2 * |X \cap Y|}{|X| + |Y|} \text{ or } Dice_Loss(X,Y) = 1 - Dice(X,Y)$$

(Figure 4B): shows the dice loss function, and the variable is the same as the Dice Coefficient, but with a the Dice Coefficient being subtracted from 1

5. *U-Net Explanation:*

U-Net, a convolutional neural network architecture widely employed for image segmentation, features a distinct structure composed of several key components. The Encoder (Contracting Path), situated at the top of the U-shaped design, is integral to the model. It incorporates a series of convolutional and pooling layers, leveraging small receptive field convolutions to capture features in the input image. The use of max pooling layers facilitates the reduction of spatial dimensions, enabling the network to progressively capture abstract and higher-level features. In contrast, the Decoder (Expanding Path) constitutes the bottom part of the architecture, housing up-sampling and concatenation operations. Up-sampling layers work to increase the spatial resolution of feature maps, recovering information lost during the pooling steps in the encoder. Additionally, concatenation is performed between up-sampled feature maps and corresponding feature maps from the encoder, allowing the integration of both low-level and high-level features. U-Net incorporates Skip Connections, a key innovation that links corresponding feature maps from the encoder to the decoder. These connections act as a shortcut for gradient flow during backpropagation, preserving fine-grained details from the early layers

and mitigating the vanishing gradient problem. Finally, the Final Layer typically consists of a convolutional layer with a sigmoid activation function. The network's output is a pixel-wise segmentation mask, where each pixel is assigned a probability value indicating the likelihood of belonging to the object of interest. The training process of U-Net involves feeding input images into the network, obtaining predicted masks, and comparing these predictions with the ground truth masks using the Dice Loss. The goal is to minimize the Dice Loss during training, which encourages the U-Net to produce accurate and precise segmentation masks that closely match the ground truth. Figure 5 explains how U-Net is incorporated into the program *(Figure 5)*. You will need to download the Python package: “unet” to use U-Net.



(Figure 5): shows the training process of U-Net goes through from each image.

Results:

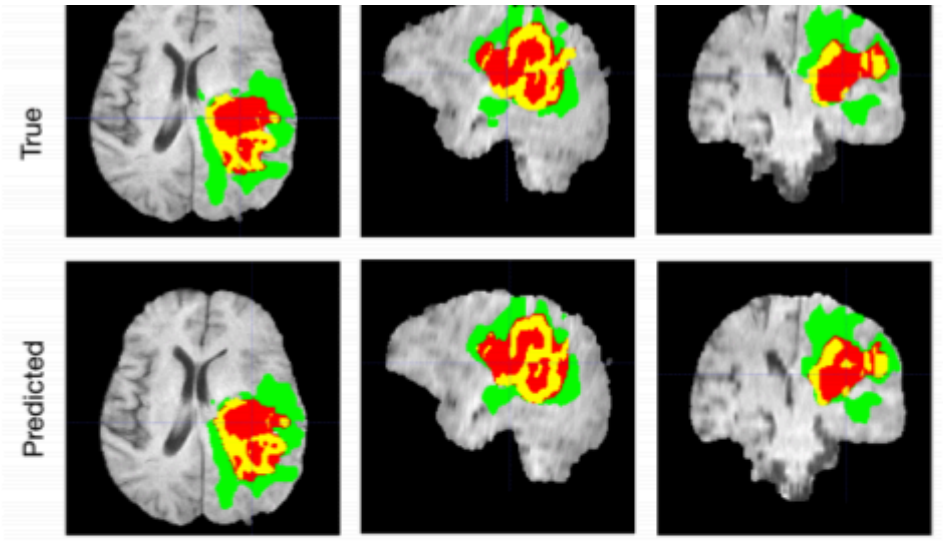
The output shows the training process of a convolutional neural network (CNN) using the U-Net architecture for image segmentation. The training is performed for 100 epochs, where each epoch involves passing the entire dataset through the network once. A total of 2,828 validated image filenames were found, which indicates the number of images used for training and validation. The model's loss, binary accuracy, Intersection over Union (IoU), and the Dice coefficient are reported for both training and validation sets. Loss is a measure of the difference between predicted and actual values, while binary accuracy calculates the proportion of correct predictions. IoU and the Dice coefficients are measures of overlap between the predicted and ground truth segmentation masks. The model reports improved validation loss (-0.02516) after the first epoch, indicating better performance on the validation set compared to the initial loss. The improved validation loss triggers the model to save itself to a file named "unet.hdf5". The training and validation loss, binary accuracy, IoU, and Dice coefficient are reported for each epoch. As the training progresses, the model's performance on the training set improves, as indicated by the decreasing loss and increasing binary accuracy, IoU, and Dice coefficient. After 10 epochs, the training loss is -0.1960, binary accuracy is 0.9833, IoU is 0.1103, and the Dice coefficient is 0.1960. These results suggest that the model has achieved reasonable performance on the training set. A UserWarning is displayed, suggesting that the HDF5 file format used for saving the model is considered legacy. The warning recommends using the native Keras format instead.

(Figure 6): Shows the output of when running the cell of the Training Parameters of the AI segmentation

Discussion:

While creating the program, the biggest issue that came across was when running the training parameters and training the AI. It took 40 minutes to complete one epoch which would mean that it would take 5 ½ days to run at the ideal epoch of 200. This problem could be resolved by upgrading the hardware on my computer. Image segmentation and Computer Vision are not fully supported and are rather slow on MacOS. This is due to the M1/M2 chip processor not fully supported for Tensorflow and with the optimizers that were used in this program. Mostafa's article mentioned in the paper managed to run the program within a few seconds. My rationale behind why he was able to train the AI quickly was because he either has a powerful Windows PC with expensive specs or he was using the newest optimizers and package versions that may work for his situation but did not work for me because Python packages and optimizers change weekly or monthly and depreciate, resulting into different results. The expected outcome was that it would take the true images from Kaggle, compare them to the user's images, and

predict where the tumor is located based on the true image (*Figure 7).* The use of computer vision and AI in medical image segmentation, particularly in the detection of tumors, holds significant implications for healthcare. Deep learning approaches, such as U-Net, have shown promising results in accurately identifying and segmenting regions of tumors in brain MRI images. The integration of computer vision and deep learning aims to create autonomous programs for tumor detection in MRI images, offering potential benefits in early diagnosis and personalized treatment plans. Overall, these advancements in medical image segmentation contribute to the broader goal of improving accuracy, efficiency, and personalized healthcare through the synergy of computer vision and AI in medical imaging.



(Figure 7): This is the result that would have shown if the training ran at a reasonable rate. It shows the true image that the AI uses, and the tumor predicted image, where the tumor that is predicted is in yellow.

Conclusion:

The objective of this research is to develop an AI program capable of segmenting and detecting tumors in Brain MRI scans, with an additional focus on enhancing the speed of the process. There have been suggestions for annotation-efficient deep-learning frameworks to enhance the diagnosis of medical images. The automation and acceleration of the segmentation process in medical imaging can be achieved through the potential of deep learning approaches. The identification and segmentation of crucial areas in medical images, such as tumors and lesions, are imperative in medical image segmentation. Obstacles encompass the requirement for an ample amount of training data, the enhancement of preprocessing techniques, and the resolution of class imbalance. The effectiveness of deep learning models is significantly influenced by the caliber of the training datasets. The main limitations encountered in the development of the program include the time-intensive training process, taking 40 minutes for one epoch, leading to a projected 5 ½ days for 200 epochs. Compatibility issues on MacOS, specifically with M1/M2 processors, contribute to slow image segmentation and Computer Vision due to incomplete TensorFlow support. The reliance on hardware and software versions, along with the dynamic nature of Python packages and optimizers, poses challenges in achieving consistent results. The author expresses a future aspiration to use a more powerful PC for quicker program execution and efficient exploration of medical image segmentation and AI applications. The overall significance of my study is to better understand the area of Image Segmentation and what is supported and what is not supported with a given piece of hardware. It also gave further insight of what optimizers and other packages future developers can use. In the future, a significantly more powerful PC than a macbook should be used to run the program in less time. The reason for this is so the images of the segmented areas of the tumors can be made.

References

Jason Walsh, Alice Othmani, Mayank Jain, Soumyabrata Dev, Using U-Net network for efficient brain tumor segmentation in MRI images, Healthcare Analytics, Volume 2, 2022, 100098, ISSN 2772-4425, https://doi.org/10.1016/j.health.2022.100098. \(https://www.sciencedirect.com/science/article/pii/S2772442522000429)

Magadza, Tirivangani, and Serestina Viriri. "Deep Learning for Brain Tumor Segmentation: A Survey of State-of-the-Art." *Journal of Imaging*, vol. 7, no. 2, Jan. 2021, p. 19. *Crossref*, https://doi.org/10.3390/jimaging7020019

Malhotra, P., Gupta, S., Koundal, D., Zaguia, A., & Enbeyle, W. (2022). Deep Neural Networks for Medical Image Segmentation. *Journal of healthcare engineering*, *2022*, 9580991. https://doi.org/10.1155/2022/9580991 (Retraction published J Healthc Eng. 2023 Oct 11;2023:9781975)

Mostafa, Almetwally M et al. "Brain Tumor Segmentation Using Deep Learning on MRI Images." *Diagnostics (Basel, Switzerland)* vol. 13,9 1562. 27 Apr. 2023, doi:10.3390/diagnostics13091562

Suganyadevi, Setal. "A review on deep learning in medical image analysis." *International journal of multimedia information retrieval* vol. 11,1 (2022): 19-38. doi:10.1007/s13735-021-00218-1

“Paul, Rohan. *Unet Brain-MRI-Segmentation | TensorFlow | Computer Vision | Deep Learning | Kaggle | Medical Image*.” 28 Apr. 2022, www.youtube.com/watch?v=X1q993i7Gic&t=4746s. Accessed 18 Oct. 2023.

"The U-Net (Actually) Explained in 10 Minutes." *www.youtube.com*, www.youtube.com/watch?v=NhdzGfB1q74&t=214s. Accessed 21 Jan. 2024.